



نظري

7

15

1800 sp

كلية الهندسة المعلوماتية

السنة الخامسة - هندسة البرمجيات

Meta Modeling

د. باسم قصيبة



RB Informatics; 25/11/2024

هندسة برمجيات ٣

تعلمنا سابقاً مصطلحي ال validation وال verification.

validation: اختبار النظام بعد بناءه.

verification: اختبار النماذج قبل أن يتم بناء النظام فإن نجحت الاختبارات نبني النظام وإلا نقوم بالتعديل على النماذج وهكذا...

أخذنا نوع من أنواع النماذج وهي شبكات بتري والتي تندرج ضمن verified system.

ولكن السؤال هل شبكات بتري هي النموذج الوحيد في ال verified system؟

الجواب لا، يوجد هناك نماذج أخرى والتي سنتناولها هذه المحاضرة.

LTL (Linear Temporal Logic)

هو منطق زمني خطي يمكن من خلاله ترميز صيغ أو أنماط تشير إلى الوقت.

هذه الصيغ أو الأنماط تتألف من روابط منطقية ($\neg$, $\wedge$, $\vee$) ومشغلات زمنية (always, sometimes, tomorrow)

الروابط المنطقية Boolean operators:

$$\begin{aligned} false &:= \neg true \\ \varphi \wedge \psi &:= \neg(\neg\varphi \vee \neg\psi) \end{aligned}$$

المشغلات الزمنية temporal operators

X (next): يحدث (تصبح محققة) في الحالة (state) التالية.

مثال: $X \varphi$

أي φ ستصبح محققة في الحالة التالية مباشرة.

G (globally, always): مُحَقَّقة دائماً.

مثال: $G \varphi$

أي أن φ محققة دائماً.



▪ F (in the future) : سوف تصبح محققة في وقتٍ ما في المستقبل.

• مثال $F \varphi$

أي أن φ ستتحقق في وقتٍ ما في المستقبل.

▪ $\cup$ (until)

• مثال $\varphi \cup \psi$

أي أن φ سيبقى مُحققة إلى أن تأتي ψ وتتحقق.

Semantics (examples)

أمثلة على خواص معينة لنظام ما:

• دائماً بعد p سوف تتحقق q .

$$G(p \Rightarrow F q)$$

• دائماً بعد p تتحقق q مباشرة بعدها (في المثال السابق q قد تتحقق بعد مُدة وليس مباشرة).

$$G(p \Rightarrow X q)$$

• في المستقبل سوف تستقر p وتصبح محققة دائماً.

$$F G p$$

• p دائماً مُحققة.

$$G p$$

Safety Properties

السلامة:

"لن يحدث شيء سيئ" / "something bad will not happen"

أمثلة:

$$G \neg (reactor_{temp} > 1000)$$

$$G \neg \left((x = 0) \wedge X \left(y = \frac{z}{x} \right) \right)$$

ملاحظة: غالباً مع خواص السلامة نستخدم $G \neg$.

Liveness Properties

الديمومة أو الاستمرارية:

"شيء جيد سوف سيحصل" / "Something good will happen"

أمثلة:

 $F \text{ rich}, F(x > 5), G(\text{start} \Rightarrow F \text{ terminate}), G(\text{trying} \Rightarrow \text{Critical})$

Fairness Properties

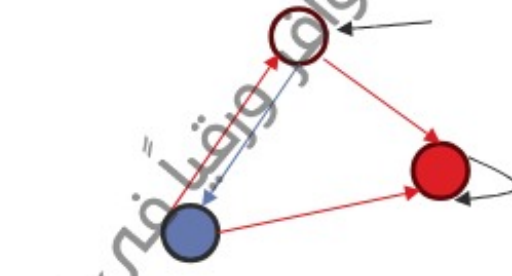
- -عادة تكون مفيدة فقط في جدولة المهام والرد على الرسائل، إلخ...
- "-إذا كان يتم محاولة أو طلب أمر ما بشكل كبير، فسوف ينجح أو يتحقق بشكل كبير".

أمثلة:

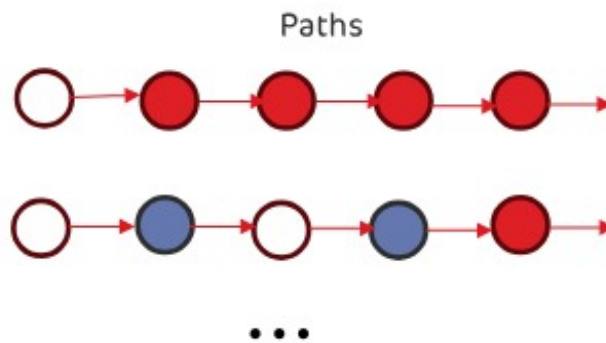
 $F \text{ ready} \Rightarrow F \text{ run}$

سؤال : هل توجد خصائص لا يستطيع LTL تمثيلها ؟

نعم مثل المخطط التالي:



حيث ستكون المسارات الممكنة كالتالي:



لذلك وجد نموذج آخر يدعى CLT.

CTL (Computational Tree Logical)

تستخدم لوصف المسارات الناتجة من حالة معطاة.

محددات المسار:

- A : مُحقق لجميع المسارات الحسابية الناتجة من الحالة.
- E : مُحقق لبعض المسارات الحسابية الناتجة من الحالة.

معاملات المشغل الزمني الخطي:

- Gp : تكون p مُحققة في كل حالة على المسار.
- Fp : تكون p مُحققة لبعض الحالات على المسار.
- Xp : تكون p مُحققة في الحالة التالية من المسار.
- $p \cup q$: تكون p مُحققة إلى أن تتحقق q في حالة ما على المسار.

1. تستخدم لغات مثل ال Promela للتحقق من نماذج CTL و LTL (غير مطلوبة).

2. يمكن تمثيل نماذج LTL و CTL كـ أوتومات.

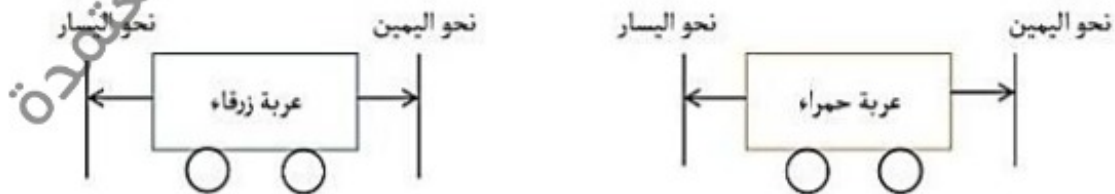
شبكات بتري صعبة في أمرين:

- النمذجة: حيث يحتاج رسمها إلى خبرة و معرفة.
- التحقق: حيث يجب رسم شجرة التغطية للتحقق من أن النظام الذي تعبر عنه شبكة بتري المرسومة صحيح.

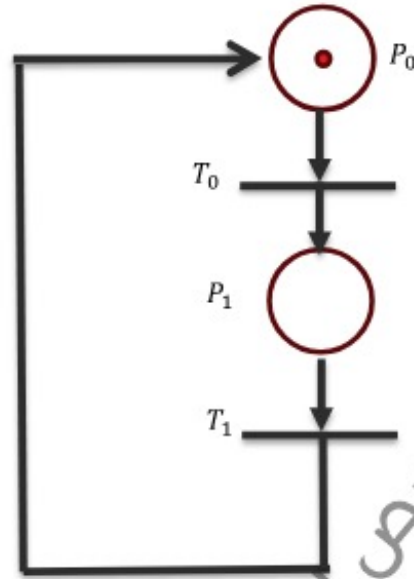
ولهذا السبب تم إيجاد نماذج مثل CTL و LTL.

شبكات بتري الملونة (غير مطلوبة في الامتحان):

مثال: لدينا نظام مؤلف من عربتين ، العربة الأولى زرقاء والثانية حمراء ، تتحرك العربة نحو اليمين أو نحو اليسار ويوجد في كلا الاتجاهين حائط ، عند وصول العربة للحائط تتحرك بالاتجاه المعاكس.

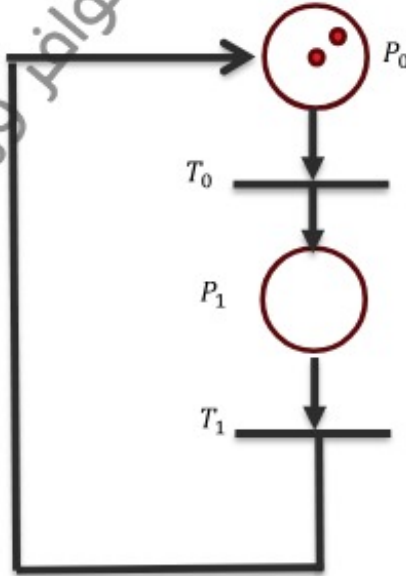


تمثيل النظام من أجل العربة الزرقاء مثلاً:



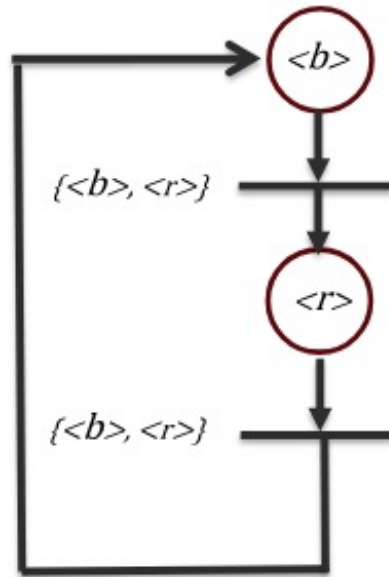
P0: تحرك نحو اليمين، T0 وصول نحو الحائط، P1 تحرك نحو اليسار، T1 وصول نحو الحائط.
العربة الحمراء كذلك الأمر نفس شبكة بناني للعربة الزرقاء.

فإذا أردنا اختصار الشبكتين بشبكة واحدة تصبح:



العلامات داخل P0 تدل على العربتين.

ولكن هذه الأخيرة لا تكافئ الشبكتين الخاصتين بكل عربة من العربتين، حيث أنه عندما ستتحرك إحدى العربتين نحو اليمين والأخرى نحو اليسار لن نستطيع تحديد من منهما العربة الزرقاء ومن منهما العربة الحمراء، يمكن حل هذه المشكلة بتحديد الألوان مثلا:



بذلك استطعنا تحديد من هي العربة المتحركة نحو اليمين وكذلك المتحركة نحو اليسار، نرى أنه قد استطعنا من خلال تحديد الألوان ضغط النظام المؤلف من أجزاء متماثلة. في شبكة بتري الملونة انتقلنا من الاهتمام بوجود العلامة فقط إلى الاهتمام بلونها أيضاً.

Meta Modeling

مقدمة: أدى تطور التقنيات والبحث العلمي إلى وصول العالم إلى مراحل متقدمة من العلم لا نستطيع السيطرة عليها، لذلك نحتاج إلى نماذج لضبط عملية التطور التقني السريع.

مراحل تطور مفاهيم التقنيات:

1. Procedures:

وهي أولى التقنيات التي ظهرت، مثل ال functions.

2. Objects:

حيث تم تطوير مفهوم procedure إلى OOP يتمثل ب object, class, method, attribute.

3. Components:

ومن أبرز الأمثلة عنها هي ال frameworks (سيتم شرح ال component لاحقاً).

4. Models:

ملاحظات:

- لا يوجد تقنية مثالية ابداً.
- التقنيات الجديدة تولد مشاكل جديدة.
- تطور التقنيات لا ولن يتوقف.

المشكلة التي نريد حلها هي: فصل منطق العمل (business logic) عن الكود التقني.

مثال:

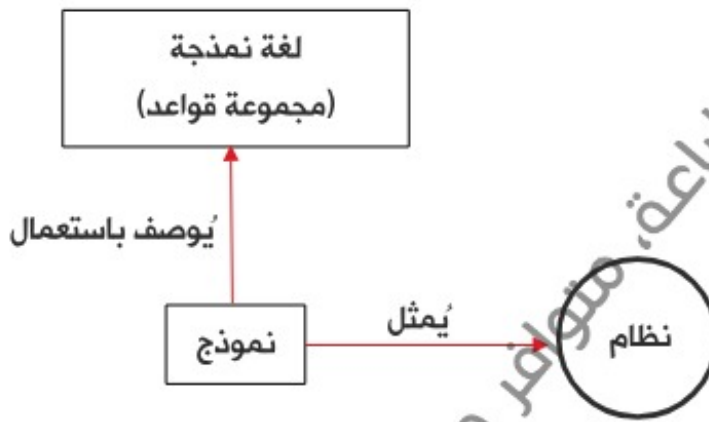
موقع تجارة إلكترونية يحوي صفحة المنتجات ويمكن للمستخدم شراء المنتج عن طريق ضغط زر add to cart أو عن طريق الصوت.

نلاحظ أن الطريقة الأولى (ضغط الزر) و الطريقة الثانية (صوت) هي طرق تقنية مختلفة لكن منطق العمل واحد (شراء المنتجات).

النموذج:

هو تجريد لشيء ما في العالم الحقيقي، يمثل النظام، بحيث يتبع عدداً من القواعد المعرفة بشكل جيد، ويجب عن الأسئلة التي تهمنا نيابة عن النظام.

يتم التعرف على النموذج باستخدام لغة نمذجة.

**مصطلحات**

- النمذجة: تصميم الأنظمة باستعمال مفاهيم وقواعد معرفة بشكل مسبق.
- Meta Modeling: عبارة عن مفاهيم لنمذجة الأنظمة أو meta-models.
- Meta-model: نموذج يُعرّف لغرض دقيق حيث يقدم مفاهيم متكيفة وأدوات متخصصة بمجال معين من التطبيقات (أي هو مجموعة المفاهيم المستخدمة لبناء نموذج وهذا النموذج سيمثل شيء بالعالم الحقيقي).

مثال :

- جدول ال Sequence Diagram: الجدول sequence هو عبارة عن meta-model لمعرفة التسلسل الزمني للنظام المدروس، و محداداته (مثل messages, actors) هي Meta Modeling.
- ال ERD: نستخدم مفاهيم table, column, relation وبالتالي نستخدم توصيف لنوع معين من النماذج وهي جداول قواعد المعطيات.
- عند عمل نمذجة للمكونات البرمجية components فإننا نستخدم component, port, connector.

MOF (Meta-Object Facility)

ال (Object Management Group) OMG تبنت ال MOF كوسيلة قياسية من أجل ال "Meta_Modeling".

- إن ال MOF مستقل عن تقنيات التنفيذ.
- إن ال MOF يسمح بالدرجة الأولى بالتعبير عن معنى ال "Meta-Model" بينما التركيب اللغوي يأتي بالدرجة الثانية.
- ال MOF يقترح منحى "Top-Down" لتصميم الأنظمة مما يسمح بتوصيف تدريجي للمشكلة ومن ثم كيفية حل المشكلة.
- ال MOF يعرف روابط بين مستوياته الثلاثة الأولى بطريقة تسمح بأتمتة تطور الأدوات.
- نستطيع معالجة المستويات المنخفضة بالاعتماد على المستويات الأعلى.

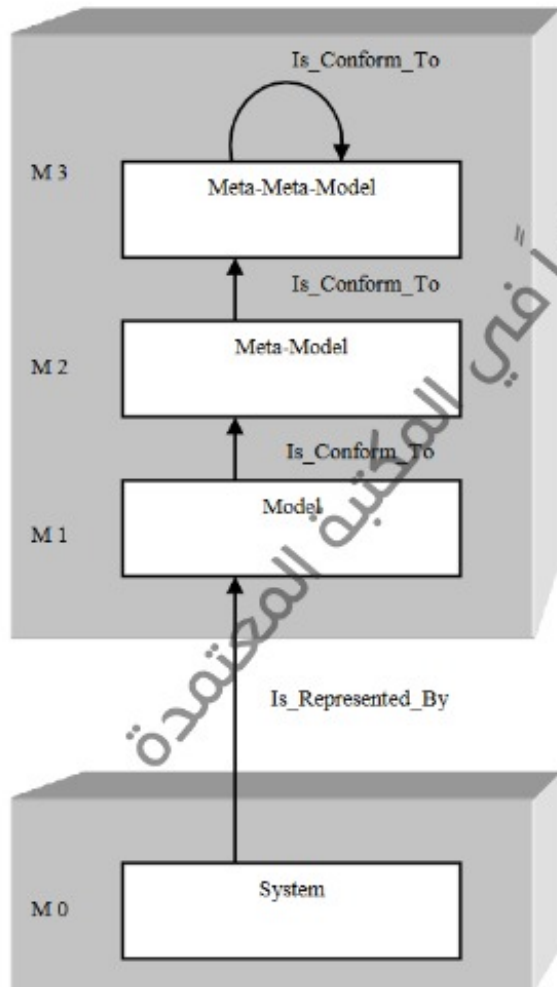
MOF Architecture

ال MOF معرف كمعمارية من أربعة مستويات M0, M1, M2, M3.

- كل مستوى يحتوي المعلومات التي تصف المستوى الأدنى.

معمارية ال MOF منظمة بالأحرى من 1+3 مستوى.

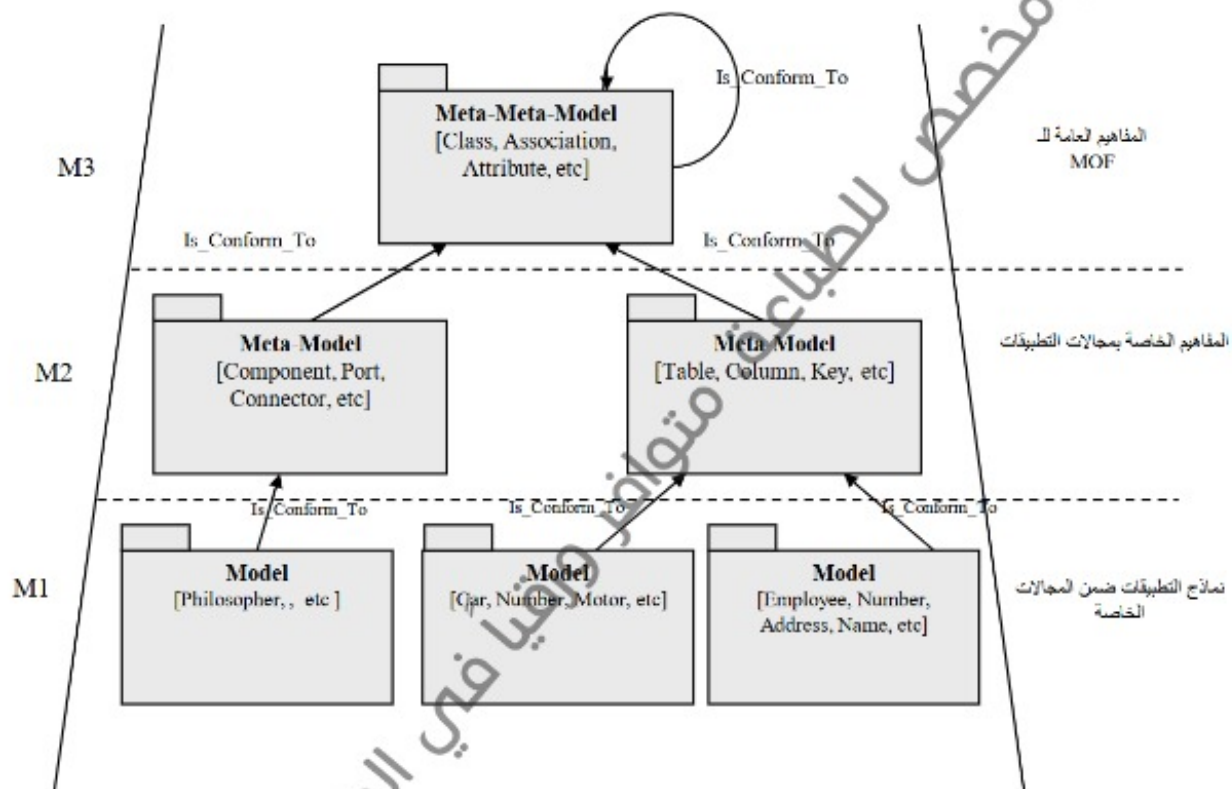
- المستوى M0 هو النظام الحقيقي، وهو ممثل بال Model المعرف عند المستوى M1.
- نموذج المستوى M1 يكون موافقاً لل Meta-Model المعرف في المستوى M2.
- ال Meta-Model يوافق ال Meta-Meta-Model عند المستوى M3.
- ال Meta-Meta-Model يوافق نفسه.



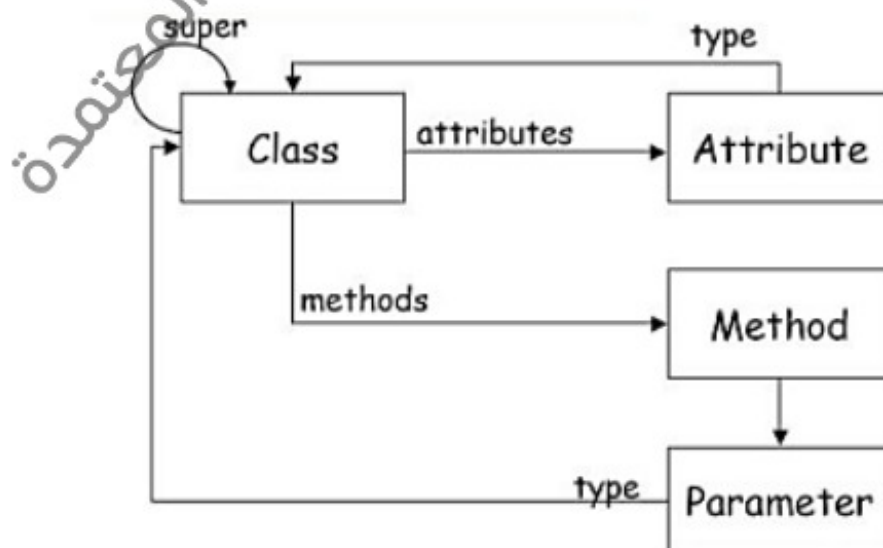
- علاقة is conform to ليست بسيطة، وتعني أن كل المفردات الموجودة بالـ model هي عبارة عن instance من المفردات الموجودة بالـ meta model ، فإنه عندما يكون لدينا ERD لها table, relation نقوم بعمل (table1) فهو غرض من table، و (table2) هو غرض من table، و R1 (وهي العلاقة بينهما) عبارة عن غرض من relation.

ملاحظة: عند ذكر MOF يكون القصد meta-meta-model.

MOF Architecture - Example

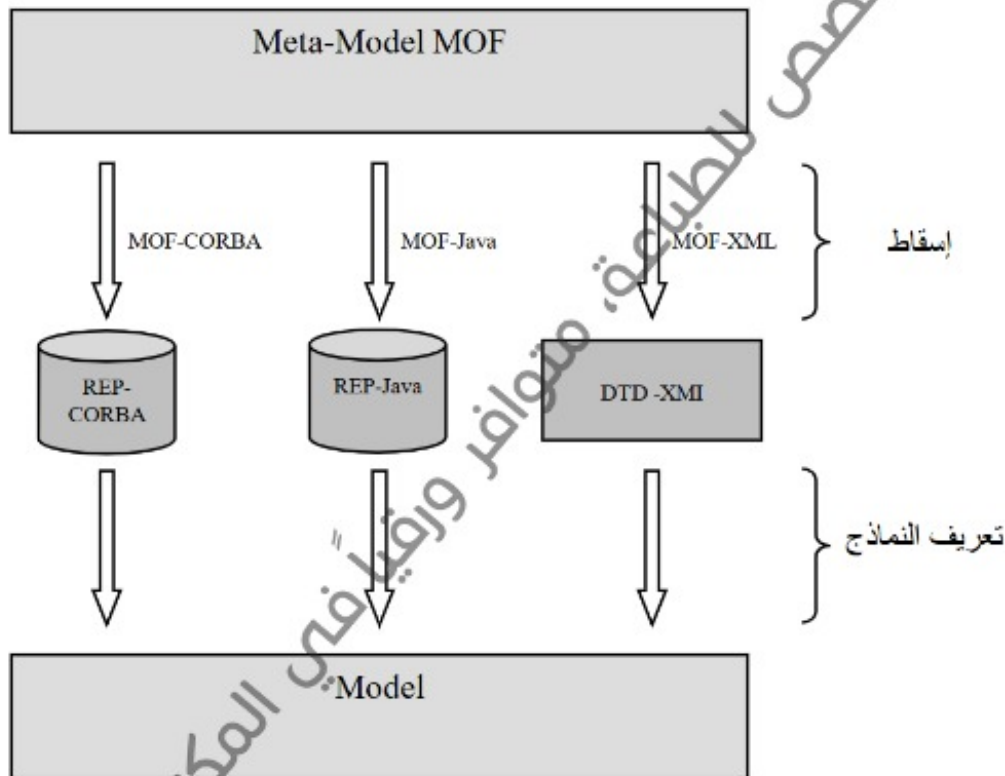


M2 for Java → Meta-programming



From MOF to tools

1. MOF يقترح مصطلح ال Repository كوسيلة لمعالجة و تخزين النماذج بشكل فيزيائي: أي أن تخزين النماذج يمكن أن يتم ضمن قاعدة بيانات أو ضمن ملف أو ضمن الذاكرة فقط.
2. ال Repository هو مجموعة من العمليات التي تسمح بمعالجة نوع من النماذج MOF.
3. يعرف فقط القواعد التي تسمح اعتباراً من Meta-Model بتوليد واجهة ال Repository.
4. ال MOF مستقل عن تقنيات التنفيذ: يعرف عدة إسقاطات نحو التقنيات الأكثر أهمية JAVA, XML, CORBA.



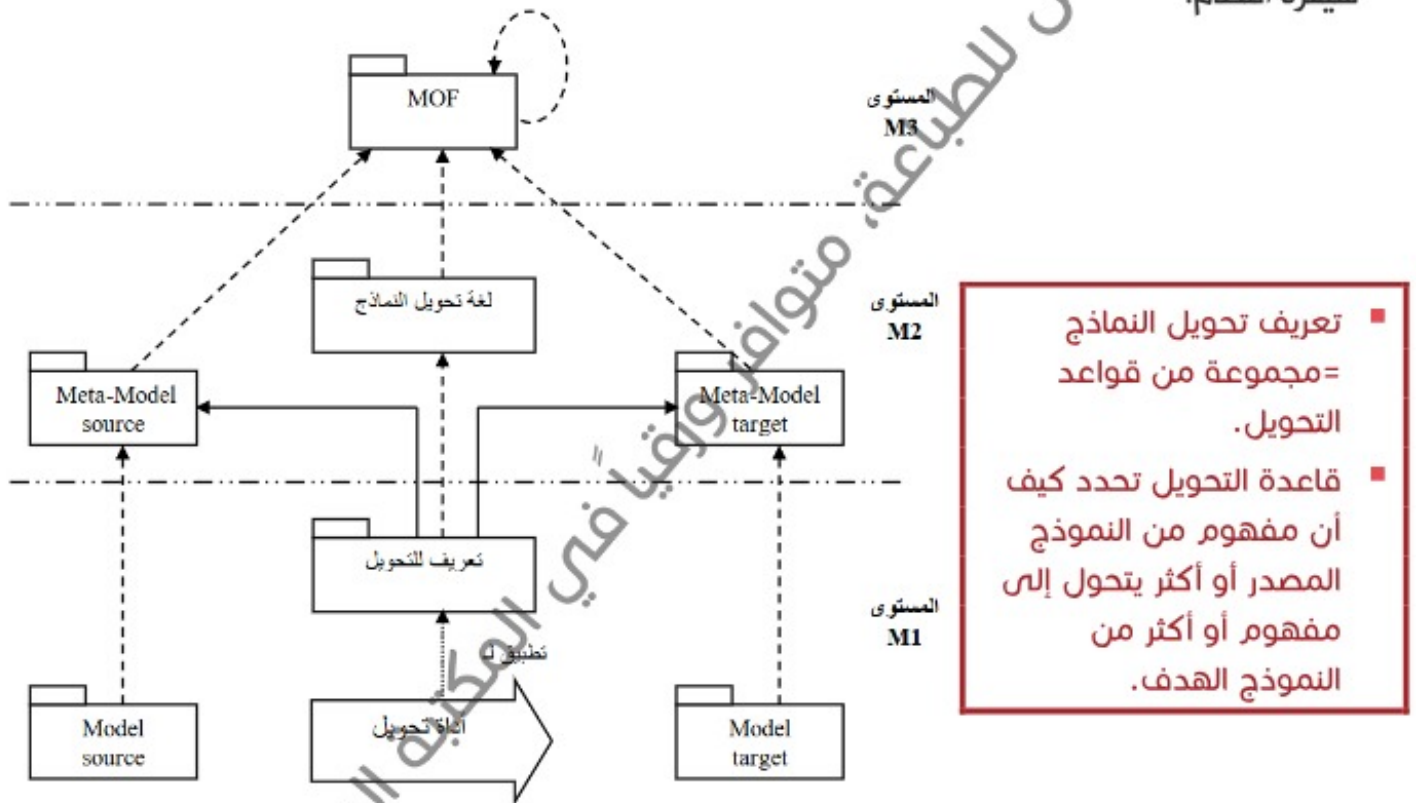
استثمار النماذج

- النماذج تسمح بفهم ومعالجة تجريد من النظام حسب وجهة نظر معينة.
- الاتجاه الحالي لهندسة البرمجيات يحاول أتمتة تطوير الأنظمة اعتباراً من النماذج التي تصف تجريداتها.
- النماذج يجب أن تكون قابلة للتفسير من قبل الحاسب.
- إمكانية تطبيق عمليات كالدماج والإسقاط على النماذج بهدف الانتقال من مستوى مجرد إلى آخر أو إلى شيفرة النظام.

تحويلات النماذج:



- يمكن امتلاك عدة نماذج تصف نفس النظام.
 - هذه النماذج تصف الأجزاء المختلفة للنظام.
 - هذه النماذج تصف النظام حسب عدة وجهات نظر.
 - على عدة مستويات تجريد مختلفة.
- بما أن عدة نماذج تناقش نفس النظام فإنه يتوجب:
 - الانتقال من نموذج مجرد إلى نموذج أقل تجريداً.
 - دمج النماذج التي تصف عدة مظاهر من النظام في نموذج واحد.
- تحويل النماذج هو العمليات المختلفة التي نطبقها على نموذج أو مجموعة نماذج بهدف إنتاج نموذج آخر أو شيفرة النظام.



تحويل النماذج: هي مجموعة من قواعد التحويل.

لدينا لغات للتحويل، معمارية وغير معمارية مثل: QVT.

عند القيام بعملية تحويل نماذج بين ال source وال target أورش منها وأستخدم مجموعة من صفاتها، وعملية تحويل النماذج واقعة في المستوى M1 ويستخدم لتوليد أداة تحويل تأخذ model source الموافق لل meta model source وينتج model target الموافق لل meta model target.

تعريف التحويل هنا هو بحد ذاته model.

أي الاتصال من نموذج لنموذج هو بحد ذاته نموذج، أما الأداة ليست ضمن M1 وإنما هي بمستوى M0.

الهندسة المقادة بالنماذج (MDE (Model Driven Engineering

- فكرة ال MDE هي استخدام النماذج وتحويلات النماذج لتنظيم فعالية تطوير نظام ما.
- MDE يسمح بشكل أفضل بالتعبير عن المشكلة، فهو يشجع على تحديد وفصل الاهتمامات المختلفة لنظام ما، مما يسمح بنمذجة كل اهتمام بشكل مستقل عن الآخر ومن ثم نعرف دمج توصيفات النظام.
- MDE يسمح بتوصيف منهجية لتعريف المشكلة وكيفية الذهاب إلى حلها، إنه يقسم فعالية تطوير البرمجيات على عدة مستويات من التجريد، النماذج في مستوى معين تختلف عن النماذج في مستوى آخر بحجتهم المهنية او التقنية، والانتقال بين هذه المستويات يتم عبر تحويلات النماذج.
- MDE يسمح برأسملة التوصيفات البدائية للنظام ورأسملة معرفة تطوير هذا النظام، معرفة تطوير النظام تُعرف ضمن تحويلات النماذج، هذا يسمح بامتلاك وثائق محدثة لشيفرة النظام.
- ميزة هامة لل MDE هي أن فعالية التطوير لبرمجية ما بإمكانها أن تتكيف بسرعة لتغيير في شروط تحقيق النظام، والآثار المترتبة لتغيير ما في شروط النظام يمكن حصرها بسهولة بينما يمكن إعادة استخدام الأجزاء الأخرى الغير مصابة.
- ال MDE يمثل تطور ضمن مجال تطوير البرمجيات حيث أنه يرفع مستوى التطوير من التجميع المباشر للأغراض إلى تحويلات النماذج، هذه التحويلات مقادة بالنماذج المعرفة في مستوى تجريد أعلى مما يسمح بأتمتة عملية تجميع الأغراض = < هكذا فإننا نتجه نحو جعل عملية تطوير البرمجيات قابلة للتصنيع.

■ أي أن MDE: هي استخدام النماذج وتحويلها لتنظيم فعالية تطوير نظام ما.

يسمح بشكل أفضل بالتعبير عن المشكلة وفصل الاهتمامات المختلفة للنظام مما يسمح بنمذجة كل اهتمام على حدى ومن ثم الدمج.

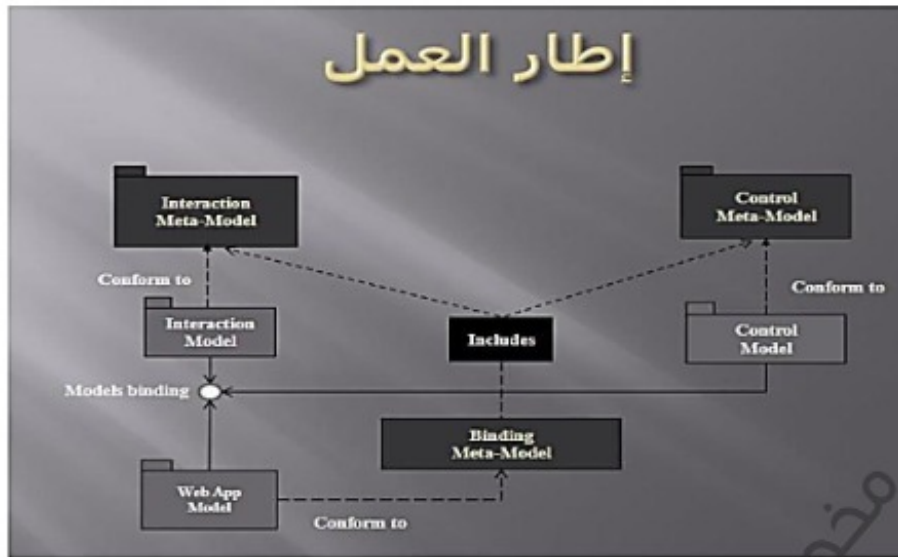
مثال: لماذا نحتاج إلى MDE؟

لأن الدنيا ليست كلها UML (غرضية التوجه)، نحن نرى أن هنالك لكل نوع من الأنظمة مفاهيمها المختلفة- فنظم الزمن الحقيقي المفهوم الأساسي لها هو synchronization task، وأما ال web app فالمفهوم الأساسي له هو الصفحة وال navigation، وبالتالي ليست دائماً نفس المفاهيم (method, class).

فليكن لدينا طريقة لبناء موقع ويب بشكل آلي، ويكون لها التسلسل الآتي:

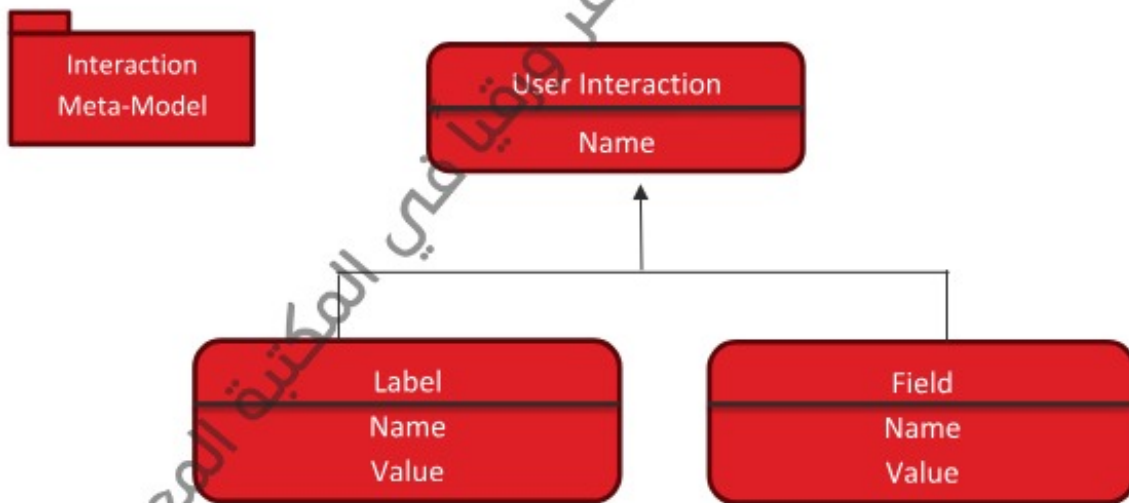
الهندسة هي تقسيم العمل الى مراحل (مثال: بدلاً من جعل عملية بناء التلفاز عند شخص واحد فقط، نقوم بعمل خط إنتاج)، هي عملية نقل الفن من تصنيع التلفاز إلى عمل خط إنتاج. وبالتالي نحن نبرمج بفن ونريد نقل هذا الفن من برمجة الويب إلى خط إنتاج لتطبيقات الويب.

إطار العمل

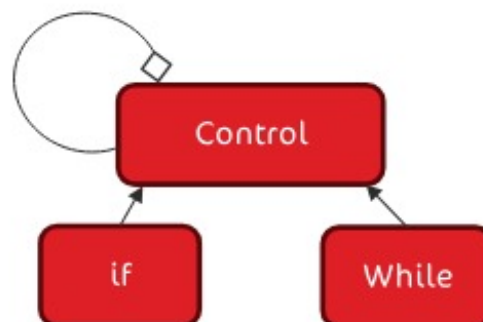


إذاً MDE هو تقسيم ال process إلى أجزاء، فلماذا لا أصمم الصفحات ومن ثم يتم الربط. تصميم الصفحات بمعزل عن اللغة، نتعرف فقط على output, input, (label, field, sound, ...). ومن ثم نعرف ال navigation ب control meta model والربط بينهما من خلال Binding meta model - عملية الدمج.

Interaction Meta-Model



Control Meta-Model لتوضيح ربط الصفحات مع بعضها:



- فكرة الدمج كانت بأن نأخذ المفاهيم الخاصة بـ Interaction Meta Model ومفاهيم Control Meta Model بالإضافة إلى مفاهيم إضافية تربط بينها web application model توضع على أساس business العمل ويظهر بعدها النموذج الجديد بعد الدمج.
- قد تم عمل أداة لـ Interaction Meta Model تستخدم لتعريف الصفحات (تعريف ثم تصدير) وهذه الأداة رسومية أما الأداة الخاصة بالـ control تستخدم لإضافة شروط للصفحات (مثل شروط على field) لم تدعم بأداة رسومية لعدم أهمية ذلك، أما Binding يتم فيه ربط الـ field مع شرطه وتم توليد أداة فيها كل المفاهيم السابقة، تعريف condition و if وهذا الـ condition من صفحات معينة.

هنا تتبين فائدة ثانية: بدلاً من أن يكون الـ control مبعثر في كل الصفحات أصبح بـ model مجرد، ويمكن توليد الصفحات بمعدل عن الـ control. ومتوفر لدى الدكتور code generator يأخذ XML ويولد صفحات php, html.

الفكرة المهمة هنا هي دمج الـ model.

خلاصة: لدمج نموذجين نأخذ مفاهيم الأول ومفاهيم الثاني مع إضافة مفاهيم تساعد في عملية الربط.

مثال: نمذجة معيار الـ CMMI:

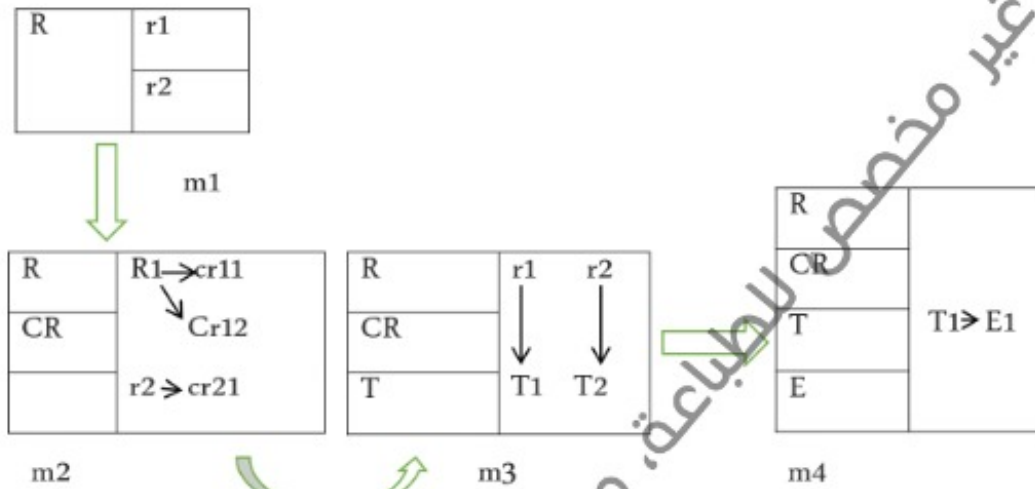
- CMMI وهو standard خاص بتطوير البرمجيات.
- Capability Maturity Model Integration تكامل نموذج نضوج القدرة.
- وهناك أداة جاهزة تسمى TFS (Team Foundation Server) من شركة مايكروسوفت له مهمة في المشاريع وتحديد المهام وإسنادها للأشخاص وفقاً لـ requirement معينة.
- وبما أن الـ TFS مكلف مادياً وزمناً كان هنالك فكرة ثانية بحيث كل شركة من الشركات تقوم بعمل إجرائية تماشي CMMI بدون أخذ أدوات معقدة مثل TFS لتطبيقها.

وبالتالي ما هي العمليات التي أريدها ؟!

- بداية نقوم بتعريف المتطلبات (Requirements) الخاصة بك (ضمن المشروع)، ومن ثم تتم عملية Review وينتج عنها change Requirement وبعد التأكد من صحة الـ Requirements نقوم بتوزيع Requirements على Tasks وكل Task لها أكثر من Requirement وكل Requirement لها أكثر من Task.
- وعند الخوض في تحليل CMMI وعملياته لا نستطيع السيطرة عليه وتكون بحاجة إلى بنية لتحميل هذه المعلومات كلها من (change requirement, Requirement)، فكانت النتيجة meta mode كبير جداً.

فتم التطبيق يدويا كالتالي:

بناء أداة تعريف Requirement وخرجها XML للمتطلبات، فنقوم بإدخاله إلى أداة ثانية لتعريف change Requirement الناتج عن عملية Review للمتطلبات، وبعدها نأخذ M2 ونخرج منه أداة المرحلة الثالثة وهي توزيع Requirements على tasks، وبالتالي تم تحويل meta model كبير إلى Sub meta models وأخذناه وولدتنا أداة.



تنفذ ال model وتأخذ الخرج لأداة ثانية، وبالتالي تم العمل على مراحل، وبالتالي بدلاً من شراء TFS (باهظ الثمن) قدمنا أداة وميزاتها عن TFS أنها يدوية في البناء بحيث يمكن أن تكون مخصصة لبيئة عمل كل شركة على حدى، ونرى هنا تطبيق MDE ليس دمج وإنما علاقات احتواء وعملية تزايدية في بناء ال model.

كانت المشكلة بالمشروع المطبق (مشروع ماستر) هي عدم وجود DB (علماً أن TFS يسمح لك ببناء واجهات ولكل واجهة DB خاص بها)، وأن هذا الأمر مهم وذلك عند القيام باستعلام يسمح لي بالحصول على معلومات عن المشروع.



اللحظة التي تجد فيها قوة
استثمرها ، انشغل بها، خذ شيئاً متراكماً وأنجزه

انتهت المحاضرة